

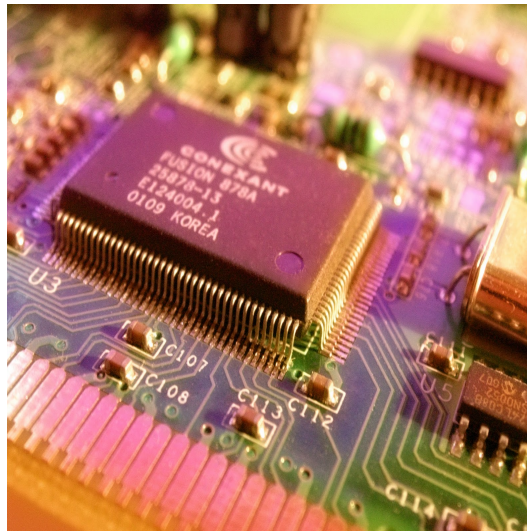
# Systèmes Embarqués (SM57)

...

## Éléments de correction des Travaux Dirigés

Alexandre RAVEY

14 avril 2023



Alexandre.Ravey@utbm.fr



**utbm**  
université de technologie  
Belfort-Montbéliard

# Décodage de trames CAN - TD N°1

En effectuant une mesure sur un bus CAN automobile avec un oscilloscope, on a relevé 3 trames présentées sur les documents réponse. Il s'agit de trames au format standard (version A).

## 1 Trame n°1

1. Compléter les cases avec la désignation (en français) du champ correspondant.
2. Quelle est la valeur de l'identifiant de la trame (en binaire et hexadécimal)? **La valeur de l'identifiant sur 11 bits est : 0x435.**
3. Compléter le tableau suivant représentant le champ de données. Il est possible que certaines cases restent vides.

| OCTET      | VALEUR BINAIRE | VALEUR HEXADÉCIMALE |
|------------|----------------|---------------------|
| 1ère octet |                |                     |
| 2ème octet |                |                     |
| 3ème octet |                |                     |
| 4ème octet |                |                     |
| 5ème octet |                |                     |

| OCTET      | VALEUR BINAIRE | VALEUR HEXADÉCIMALE |
|------------|----------------|---------------------|
| 1ère octet | 10001101       | 8D                  |
| 2ème octet | 01011000       | 58                  |
| 3ème octet | 11010010       | D2                  |
| 4ème octet |                |                     |
| 5ème octet |                |                     |

## 2 Trame n°2

1. Quelle est la valeur de l'identifiant de la trame (en binaire et hexadécimal)? **La valeur de l'identifiant sur 11 bits est : 0x561.**
2. Compléter le tableau suivant représentant le champ de données. Il est possible que certaines cases restent vides.

| OCTET      | VALEUR BINAIRE | VALEUR HEXADÉCIMALE |
|------------|----------------|---------------------|
| 1ère octet |                |                     |
| 2ème octet |                |                     |
| 3ème octet |                |                     |
| 4ème octet |                |                     |
| 5ème octet |                |                     |

| OCTET      | VALEUR BINAIRE | VALEUR HEXADÉCIMALE |
|------------|----------------|---------------------|
| 1ère octet | 01001010       | 4A                  |
| 2ème octet | 10000000       | 80                  |
| 3ème octet | 10110001       | B1                  |
| 4ème octet |                |                     |
| 5ème octet |                |                     |

### 3 Trame n°3

1. Placer correctement les pointillés qui permettent de délimiter les champs.
2. Quelle est la valeur de l'identifiant de la trame (en binaire et hexadécimal)? **La valeur de l'identifiant sur 11 bits est : 0x7C3.**
3. Compléter correctement la zone en pointillés.
4. Compléter le tableau suivant représentant le champ de données. Il est possible que certaines cases restent vides.

| OCTET      | VALEUR BINAIRE | VALEUR HEXADECIMALE |
|------------|----------------|---------------------|
| 1ère octet |                |                     |
| 2ème octet |                |                     |
| 3ème octet |                |                     |
| 4ème octet |                |                     |
| 5ème octet |                |                     |

| OCTET      | VALEUR BINAIRE | VALEUR HEXADECIMALE |
|------------|----------------|---------------------|
| 1ère octet | 10000001       | 81                  |
| 2ème octet | 10101100       | AC                  |
| 3ème octet |                |                     |
| 4ème octet |                |                     |
| 5ème octet |                |                     |

### 4 Etude d'un réseau CAN

Sur un véhicule automobile est installé un réseau CAN 2.0A. Les différents noeuds sont les suivants :

- Noeud 1320 : calculateur moteur (variables concernant le moteur)
- Noeud 7020 : calculateur ABS (variables concernant les freins et les différentes vitesses de roue et de véhicule)
- Noeud 0004 : combiné d'instrumentation (affichage conducteur)
- Noeud CV00 : module de commutation sous volant de direction (principaux organes de commande : clé, phare, essuie-glace, etc ...)
- Noeud 6000 : calculateur de porte (état des ouvrants)
- Noeud 1630 : calculateur boîte de vitesse automatique
- Noeud BSI : boîtier principal

Voici une liste de quelques paramètres circulant sur le réseau :

- Vitesse de rotation du moteur (identifiant 0x3C1 : gamme 0..7650 tr/min sur 8 bits).
  - Vitesse du véhicule donnée par le calculateur ABS (identifiant 0x7D9) ; gamme 0..255 km/h (vitesse maximale) sur 8 bits).
  - Vitesse des roues (en tr.min-1) données par le calculateur ABS (identifiants 0x03F ; ordre des données : avant droit, AV G, AR D, AR G : gamme vitesse maximale sur 7 bits ; le 8ème bit de poids fort indique le sens de rotation : 0 arrière, 1 avant). Diamètre de la roue = 570 mm.
1. Quelles sont les résolutions des variables numériques traduisant la vitesse de rotation du moteur, la vitesse de rotation des roues, et de la vitesse du véhicule? Détailler les calculs.  
**Vitesse rotation moteur :**  
**de 0 à 7650 tr/min sur 8 bits**  
 **$8 \text{ bits} = 2^8 - 1 = 255 \text{ Valeur}_{max}$**

$$\frac{7650}{255} = 30 \text{ tr/min/bit}$$

Vitesse du véhicule :

de 0 à 255 km/h sur 8 bits

$$\frac{255}{255} = 1 \text{ km/h/bit}$$

Vitesse des roues :

de 0 à ? tr/min sur 7 bits + 1 bit de sens

$$\Omega = \frac{V}{2 \cdot \pi \cdot R} = \frac{255 \cdot 1000 / 60}{2 \cdot 3.1416 \cdot 0.57 / 2} = 2373.36 \text{ tr/min}$$

$$\frac{2373.36}{127} = 18.69 \text{ tr/min/bit}$$

2. Donner les trames AVEC BIT STUFFING ; le champ CRC sera rempli par des croix (X) sans tenir compte des bits stuffing. On part du principe qu'il n'y a jamais d'erreur de transmission. Indiquer de couleur différente les bits de stuffing. Détailler les calculs.

— Trame indiquant une vitesse du véhicule de 62 km/h.

$$\frac{62}{R_{vehicule}} = (62)_{10} \text{ voir trame corrigée identifiant } 0x7D9 \text{ donnée } (62)_{10} = 0x3E$$

— Trame indiquant la vitesse des roues du véhicule (marche avant) pour 48 km/h avec un patinage sur la roue avant gauche qui tourne à 1650 tr/min.

$$\frac{1650}{R_{resoroues}} = \frac{1650}{18.69} = (88.28)_{10} = (88)_{10} \text{ voir trame corrigée identifiant } 0x03F \text{ donnée } (88)_{10} = 0x58 + \text{sens} = 0xD8 \text{ roue avant gauche.}$$

$$\Omega = \frac{V}{2 \cdot \pi \cdot R} = \frac{48 \cdot 1000 / 60}{2 \cdot 3.1416 \cdot 0.57 / 2} = 446.75 \text{ tr/min}$$

$$(23.9)_{10} = 0x17 + \text{sens} = 0x97$$

— Trame indiquant la vitesse de rotation du moteur pour un véhicule roulant à 85 km/h avec un rapport de pont et de boîte K équivalent de 0,28 (vitesse roue = K x vitesse moteur).

Calcul de la vitesse de rotation du moteur : Pour ce faire, on calcule dans un premier temps la vitesse de rotation des roues :

$$\Omega_{roues} = \frac{V}{2 \cdot \pi \cdot R} = \frac{85 \cdot 1000 / 60}{2 \cdot 3.1416 \cdot 0.57 / 2} = \frac{1416.66}{1.79} = 791.43 \text{ tr/min}$$

$$V_{moteur} = \frac{\Omega_{roues}}{K} = \frac{791.43}{0.28} = 2826.53 \text{ tr/min}$$

Calcul de la donnée de la trame :

$$\frac{V_{moteur}}{R_{resomoteur}} = \frac{2826.53}{30} = 94.21_{10} = 0x5E$$

Voir trame corrigée identifiant 0x3C1 donnée 94<sub>10</sub> = 0x5E

3. Dessinez les différentes trames qui seront présentes tour à tour sur le bus CAN en fonction des différents cas ; indiquer la référence du noeud qui " parle " sur le réseau. Le champ CRC sera rempli par des croix (X) :

— Le combiné d'instrumentation réclame l'information de vitesse véhicule sachant que la voiture roule à 115 km/h. Le noeud concerné répond.

La vitesse du véhicule a pour identifiant :  $0x7D9$ . Une vitesse de  $115\text{ km/h}$  correspond à une donnée de la trame de :  $(115)_{10} = 0x73$

Il faut donc dessiner :

Une trame avec  $ID = 0x7D9$ ,  $RTR = 1$  (trame de requête) provenant du noeud 0004.

Une trame avec  $ID = 0x7D9$ ,  $RTR = 0$  (trame de donnée) provenant du noeud 7020.

- Le BSI réclame l'information des vitesses des roues au boîtier ABS sachant que la voiture roule à  $28\text{ km/h}$  en ligne droite en marche arrière. Le noeud concerné répond.

L'identifiant concernant la vitesse des roues est :  $0x03F$ .

Calculons la vitesse des roues :

$$\Omega_{roues} = \frac{V}{2 \cdot \pi \cdot R} = \frac{28 \cdot 1000 / 60}{2 \cdot 3.1416 \cdot 0.57 / 2} = \frac{466.66}{1.79} = 260.7\text{ tr/min}$$

Le véhicule roulant en marche arrière, la donnée à transmettre dans la trame pour chaque roue est donc :

$$\frac{260.7}{R_{reso_{roues}}} = \frac{260.7}{18.69} = (13.95)_{10} = (14)_{10} = 0x0E + \text{sens} = 0x0E$$

Il faut donc dessiner :

Une trame avec  $ID = 0x03F$ ,  $RTR = 1$  (trame de requête) provenant du noeud BSI.

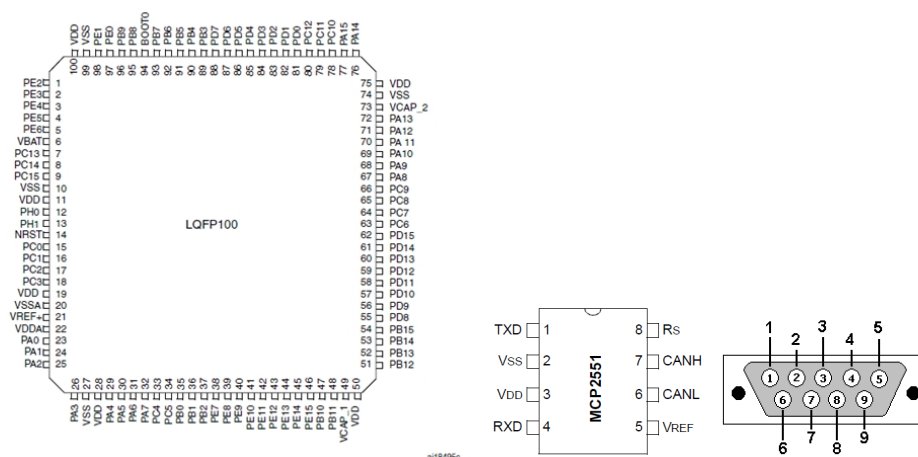
Une trame avec  $ID = 0x03F$ ,  $RTR = 0$  (trame de donnée) et 4 données similaires de  $0x0E$  provenant du noeud 7020.

# Utilisation du CAN sur STM32F407 - TD N°2

Cet énoncé de travaux dirigés permet d'acquérir les connaissances liées à l'implantation d'un nœud CAN sur un réseau. Il traite les aspects matériels (électronique) et logiciels (programmation du contrôleur en langage C) de manière à assurer une mise en œuvre ordonnée et optimisée du protocole CAN à partir d'exemples simples appliqués au micro-contrôleur STM32F407.

## 5 Connexion d'un nœud au réseau

1. En vous aidant des documentations du STM32F407 et du driver CAN MCP2551, compléter le schéma ci-dessous pour assurer la connexion au bus CAN du micro-contrôleur en mode High Speed (Standard 2.0A).



2. Indiquer à quels endroits le bus CAN doit être équipé de résistances de terminaison et donner leur valeur.

2 résistances de terminaison de valeur 120 Ohms doivent être placées à chaque extrémité du bus.

3. Rappeler les niveaux de tension des broches TXD et RXD du MCP2551 lors du transfert de trames CAN, ainsi que les niveaux électriques correspondant des broches CANL et CANH.

Les broches TXD et RXD ont des niveaux de tension de l'ordre de 0-3.3 V. Les broches CANL et CANH varie entre 1.5 V et 2.5 V pour l'une et 2.5 V et 3.5 V pour l'autre.

## 6 Configuration du Bit Timing

1. Déterminer les différents paramètres des registres de configuration du Bit Timing pour définir la configuration suivante : (APB1 à 168 Mhz)

- Baud Rate = 250kb/s
- Segment BT1 = 6 TQ
- Segment BT2 = 3 TQ
- Sauts de resynchronisation = 1 TQ
- Mode normal

En regardant la documentation du bit timing dans le cours :

$$1 NBT = 1 * t_q + BT1 + BT2 = 1 * t_q + 6 * t_q + 3 * t_q = 10 * t_q$$

$$t_q = BRP + 1 * t_{PCLK}$$

Or avec  $250\text{ kbits/s}$ , on a  $250000\text{ NBT}$  en  $1\text{ sec}$ , donc :  $BRP = \frac{F_{APB1}}{250000 * 10} - 1 = 66_{10} = 0x42$

On peut donc configurer le registre (page 90 du cours)

$CAN1\_BTR = 0x00250042$

- Déterminer la durée maximale de transmission d'une trame CAN 2.0A comportant des données de 8 octets.

En regardant le schéma page 30 du cours, la longueur maximum d'une trame est de  $44 + 8N$  avec  $N$  le nombre d'octet : on obtient donc :  $N_{bit\_MAX} = 44 + 8 * 8 = 108\text{ bits}$

Attention néanmoins, il faut prendre en compte les bits de Stuffings! en supposant que la trame est uniquement composée de 1 ou de 0 (impossible), on aura :

$$N_{stuffing} = \frac{34 + 8 * 8}{5} = 19\text{ bits}$$

On en déduit :

$$N_{bit\_MAX} = 108 + 19 = 127\text{ bits}$$

La vitesse étant de  $250\text{ kbits/s}$ , on en déduit donc :

$$T_{max} = \frac{127}{250000} = 508\text{ }\mu\text{s}$$

- Déterminer le débit utile maximal en octets/s dans le cas de cette configuration

Dans la trame de  $127\text{ bits}$ , seul  $64\text{ bits}$  sont utiles. On a donc un débit de :

$$\frac{64}{0.000508} = 125984\text{ bits/s} = 15748\text{ o/s} = 15.37\text{ ko/s}$$

## 7 Configuration du filtrage des messages

- On désire permettre au nœud CAN de réceptionner les messages dont les identifiants sont compris entre  $0x100$  et  $0x10F$  (trames de données seulement) ainsi que les identifiants  $0x200$  (trame de donnée seulement) et  $0x205$  (trame de requete seulement). Déterminer les registres de filtrage et les masques de filtrage pour affecter ces groupes d'identificateurs au FIFO0.

```

1  ////////////////////////////////////////////////////
2  //                               Filter:                               //
3  // We will use filter 0 for 0x100 to 0x10F (Mask mode) //
4  // Filter 1 for 0x200 and 0x205 (ID List mode)         //
5  ////////////////////////////////////////////////////
6
7  //Deactivate Filter 0 and 1
8  CAN1->FA1R &= 0xFFFFFFF0;
9  //initialize mode for all filters
10 CAN1->FMR |= 0x1;
11
12 ////////////////////////////////////////////////////
13 //Filters with 32 bits mode//
14 ////////////////////////////////////////////////////
15
16 CAN1->FS1R |= 0x00000003; // 32 bits filter for filter 0,1

```

```

17 CAN1->FM1R |= 0x00000002; // Filter 1 in List mod
18 CAN1->FM1R &= 0xFFFFFFF0; // Filter 0 in mask mod
19 CAN1->FFA1R &= 0xFFFFFFF0; // Assign filter 0,1 to FIFO0
20
21 //Configure the filter bank
22 CAN1->sFilterRegister[0].FR1 = (0x100 << 21); //ID
23 CAN1->sFilterRegister[0].FR2 = (0x7F0 << 21); // Mask
24
25 CAN1->sFilterRegister[1].FR1 = (0x200 << 21); //ID
26 CAN1->sFilterRegister[1].FR2 = (0x205 << 21); // ID
27 CAN1->sFilterRegister[1].FR2 |= 0x2; //ID with RTR = 1
28
29
30 //Filters with dual 16 bits config//
31
32
33
34 CAN1->FS1R &= 0xFFFFFFF0; // Dual 16 bits filter for filter 0,1
35 CAN1->FM1R |= 0x00000002; // Filter 1 in List mod
36 CAN1->FM1R &= 0xFFFFFFF0; // Filter 0 in mask mod
37 CAN1->FFA1R &= 0xFFFFFFF0; // Assign filter 0,1 to FIFO0
38
39 //Configure the filter bank
40 CAN1->sFilterRegister[0].FR1 = (0x100 << 5); //ID
41 CAN1->sFilterRegister[0].FR1 = ((0x7F0 <<5) <<16); // Mask
42
43 CAN1->sFilterRegister[1].FR1 = (0x200 << 5); // ID
44 CAN1->sFilterRegister[1].FR1 = ((0x205 <<5) <<16); // ID
45 CAN1->sFilterRegister[1].FR1 |= ((0x1 <<4) <<16); // ID with RTR=1
46 CAN1->sFilterRegister[1].FR2 = (0x200 << 5); // ID
47 CAN1->sFilterRegister[1].FR2 = ((0x205 <<5) <<16); // ID
48 CAN1->sFilterRegister[1].FR2 |= ((0x1 <<4) <<16); // ID with RTR=1
49
50 //Activate Filter 0 and 1
51 CAN1->FA1R |= 0x00000003;
52 // Leave Filter init
53 CAN1->FMR &= 0xFFFFFFF0;

```

2. Déterminer les paramètres des registres de configuration des interruptions pour autoriser le déclenchement d'interruptions sur les événements suivants :
  - Réception de trames dans FIFO0
  - Fin de transmission d'une trame par la mailbox TX0

```

1 //Interrupts
2 // Interrupts
3
4
5 //Set Interrupt RX FIFO0 (FMPIE0) and TX box empty
6 CAN1->IER |= 0x3;
7
8 ///Activate Interrupt on NVIC
9 HAL_NVIC_SetPriority(CAN1_RX0_IRQn, 0, 1);
10 HAL_NVIC_EnableIRQ(CAN1_RX0_IRQn);
11
12 HAL_NVIC_SetPriority(CAN1_TX_IRQn, 0, 2);
13 HAL_NVIC_EnableIRQ(CAN1_TX_IRQn);

```

3. Écrire en langage C la fonction `void init_CAN1(void)` permettant de configurer le contrôleur CAN avec les paramètres donnés dans les précédentes questions.

```

1 void init_CAN1(void){
2
3     GPIO_InitTypeDef GPIO_InitStructure = {0};
4
5     //Enable CAN CLock
6     __HAL_RCC_CAN1_CLK_ENABLE();
7
8     //Configure GPIOs
9     __HAL_RCC_GPIOB_CLK_ENABLE();
10    /**CAN1 GPIO Configuration
11    PB8      -> CAN1_RX
12    PB9      -> CAN1_TX
13    */
14    GPIO_InitStructure.Pin = GPIO_PIN_8|GPIO_PIN_9;
15    GPIO_InitStructure.Mode = GPIO_MODE_AF_PP;
16    GPIO_InitStructure.Pull = GPIO_NOPULL;
17    GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
18    GPIO_InitStructure.Alternate = GPIO_AF9_CAN1;
19    HAL_GPIO_Init(GPIOB, &GPIO_InitStructure);
20
21
22    //Clear Sleep bit to wakeup
23    CAN1->MCR &= 0xFFFFFFF0;
24    //Wait for CAN to wakeup
25    while(CAN1->MSR & 0x00000002);
26    //Switch to config mode
27    CAN1->MCR |= 0x00000001;
28    //Wait for config mod
29    while(!(CAN1->MSR & 0x1));
30
31    //Set config
32    CAN1->MCR &= 0x00000001;
33
34    //Set bit Timing: 250Kbauds/sec / Mode Normal / Prescaler = 66 / BS1 = 6
35    //TQ / BS2 = 3TQ
36    CAN1->BTR = 0x00250042;
37
38    //Release Mailbox (already empty?)
39    CAN1->RF0R |= CAN_RF0R_RFOM0;
40
41    ////////////////////////////////////
42    // Interrupts //
43    ////////////////////////////////////
44
45    //Set Interrupt RX FIFO0 (FMPIE0) and TX box empty
46    CAN1->IER |= 0x3;
47
48    ///Activate Interrupt on NVIC
49    HAL_NVIC_SetPriority(CAN1_RX0_IRQn, 0, 1);
50    HAL_NVIC_EnableIRQ(CAN1_RX0_IRQn);
51
52    HAL_NVIC_SetPriority(CAN1_TX_IRQn, 0, 2);
53    HAL_NVIC_EnableIRQ(CAN1_TX_IRQn);

```

```

54 // Go to normal mode
55 CAN1->MCR &= 0xFFFFFFF0;
56 //Wait for normal mod
57 while(CAN1->MSR & 0x1);
58
59 //Deactivate Filter 0 and 1
60 CAN1->FA1R &= 0xFFFFFFF0;
61 //initialize mode for all filters
62 CAN1->FMR |= 0x1;
63
64 //Filters with 32 bits mode//
65
66
67
68 CAN1->FS1R |= 0x00000003; // 32 bits filter for filter 0,1
69 CAN1->FM1R |= 0x00000002; // Filter 1 in List mod
70 CAN1->FM1R &= 0xFFFFFFF0; // Filter 0 in mask mod
71 CAN1->FFA1R &= 0xFFFFFFF0; // Assign filter 0,1 to FIFO0
72
73 //Configure the filter bank
74 CAN1->sFilterRegister[0].FR1 = (0x100 << 21); //ID
75 CAN1->sFilterRegister[0].FR2 = (0x7F0 << 21); // Mask
76
77 CAN1->sFilterRegister[1].FR1 = (0x200 << 21); //ID
78 CAN1->sFilterRegister[1].FR2 = (0x205 << 21); // ID
79 CAN1->sFilterRegister[1].FR2 |= 0x2; //ID with RTR = 1
80
81 //Activate Filter 0 and 1
82 CAN1->FA1R |= 0x00000003;
83 // Leave Filter init
84 CAN1->FMR &= 0x0;
85
86
87
88
89 }

```

## 8 Émission de trames

Pour faciliter la manipulation des messages échangés par le bus CAN, on définit une structure et un type de donnée propre aux messages CAN de la manière suivante :

```

1 typedef struct{
2     uint16_t STDID;           // ID standard (11 bits)
3     uint8_t RTR;             // Request ou Data frame
4     uint8_t DLC;             // nombre d'octets de données utiles
5     uint8_t data[8];         // données
6 }CAN_frame;

```

Il est ainsi possible de déclarer des variables de type *CAN\_frame* (trame CAN) comme ceci :

```
CAN_frame CAN_message;
```

1. Écrire en langage C la fonction *unsigned char CAN\_sendFrame(CAN\_frame CAN\_mes)* qui envoie la trame CAN donnée en paramètre à l'aide de la boîte d'envoi TX0.

```
1 unsigned char CAN_sendFrame(CAN_frame CAN_mes){
```

```

2 //Check if TxMailbox0 is empty
3 if (CAN1->TSR & 0x04000000)
4 {
5     // fill the mailbox
6     CAN1->sTxMailBox[0].TIR = (CAN_mess.STDID << 21) | (CAN_mess.RTR
7         << 1);
8     CAN1->sTxMailBox[0].TDTR = CAN_mess.DLC;
9     CAN1->sTxMailBox[0].TDLR = CAN_mess.data[3] << 24 | CAN_mess.data
10         [2] << 16 | CAN_mess.data[1] << 8 | CAN_mess.data[0];
11     CAN1->sTxMailBox[0].TDHR = CAN_mess.data[7] << 24 | CAN_mess.data
12         [6] << 16 | CAN_mess.data[5] << 8 | CAN_mess.data[4];
13     CAN1->sTxMailBox[0].TIR |= 1; //Send the message
14     return(1);
15 }
16 else return(0);
17 }

```

2. Indiquer quelle amélioration apporterait l'utilisation de l'interruption déclenchée sur fin de transmission réussie (sans écrire la fonction d'interruption).

La fonction d'interruption permet de pouvoir envoyer les trames les unes après les autres en optimisant le fonctionnement du périphérique (pas d'attente entre 2 envois) .

## 9 Réception de trames

Les trames reçues sont lues dans la fonction d'interruption signalant la réception d'une nouvelle trame.

1. Indiquer quel est l'intérêt du mode de réception par interruption décrit ci-dessus par rapport à une lecture des messages reçus sans utiliser les interruptions.

La fonction d'interruption permet de ne pas bloquer le programme principal en attente de la trame (scrutation).

2. Écrire en langage C la fonction d'interruption `void CAN1_RX0_IRQHandler(void)` qui récupère la trame et la renvoie au programme principal via une variable globale.

```

1 extern CAN_frame CAN_RxMessage;
2
3 void CAN1_RX0_IRQHandler(void)
4 {
5     //Receive CAN frame
6     CAN_RxMessage.STDID = CAN1->sFIFOMailBox[0].RIR >> 21;
7     CAN_RxMessage.RTR = (CAN1->sFIFOMailBox[0].RIR >> 1) & 0x1;
8     CAN_RxMessage.DLC = CAN1->sFIFOMailBox[0].RDTR & 0xF;
9     CAN_RxMessage.data[0] = CAN1->sFIFOMailBox[0].RDLR & 0xFF;
10    CAN_RxMessage.data[1] = (CAN1->sFIFOMailBox[0].RDLR >> 8) & 0xFF;
11    CAN_RxMessage.data[2] = (CAN1->sFIFOMailBox[0].RDLR >> 16) & 0xFF;
12    CAN_RxMessage.data[3] = (CAN1->sFIFOMailBox[0].RDLR >> 24) & 0xFF;
13    CAN_RxMessage.data[4] = CAN1->sFIFOMailBox[0].RDHR & 0xFF;
14    CAN_RxMessage.data[5] = (CAN1->sFIFOMailBox[0].RDHR >> 8) & 0xFF;
15    CAN_RxMessage.data[6] = (CAN1->sFIFOMailBox[0].RDHR >> 16) & 0xFF;
16    CAN_RxMessage.data[7] = (CAN1->sFIFOMailBox[0].RDHR >> 24) & 0xFF;
17
18    //Reset FIFO0
19    CAN1->RF0R |= 0x20;
20    //Reset interrupt
21    HAL_CAN_IRQHandler(&hcan1);

```

<sup>22</sup> }

---

# Annexes - TD N°3

## 1 Trames CAN TD4

